

Structured Readout Scheduling and Policy Design for Sparse Query Evaluation

Dan Hawkley

Abstract

Evaluating large numbers of sparse test queries is a bottleneck in data-intensive and hybrid classical–quantum workflows. While prior work emphasizes input reduction and model construction, comparatively less attention has been given to how inference itself is structured. We introduce a framework for structured readout scheduling, coverage-aware inference, and multi-objective policy design. We show that predictive performance can saturate before full coverage is achieved, that enforcing coverage constraints introduces a controllable evaluation cost, and that Pareto-efficient policies characterize tradeoffs between query cost, predictive behavior, coverage, and distributional stability. This work does not modify Quantum Oracle Sketching (QOS) or claim quantum advantage; it develops a classical inference-control layer compatible with QOS-style pipelines and other sparse evaluation settings.

1 Introduction

Many modern computational systems must evaluate large sets of candidate queries under constrained resources. This arises in large-scale machine learning inference, retrieval systems, and hybrid classical–quantum workflows such as Quantum Oracle Sketching (QOS) [3]. Most optimization effort focuses on reducing input size or improving model construction. Here we focus on a complementary question: how should many sparse queries be evaluated efficiently and systematically?

We formalize readout as a sequence of partial evaluations under a constrained query budget. At each stage, a policy selects which queries to evaluate next and determines whether evaluation should continue. This work introduces readout as a controllable component of inference, enabling structured scheduling, coverage-aware stopping, and policy optimization.

2 Problem Setting

Let $\mathcal{Q} = \{q_i\}_{i=1}^N$ denote a set of test queries. A readout policy defines an ordering or partial ordering over query indices. Let π denote a scheduling policy, and let $\pi_{1:k}$ denote the first k queries selected by that policy.

At each evaluation step, we measure predictive behavior, query cost, distributional shift, and structural coverage. The goal is not only to maximize accuracy, but to control the tradeoff between evaluation cost and structural completeness.

3 Scheduling Strategies

We evaluate four families of scheduling strategies.

3.1 Random Scheduling

Random scheduling selects query indices uniformly at random. It provides an unbiased baseline but lacks deterministic reproducibility and structural coverage guarantees.

3.2 Modular Scheduling

Modular scheduling partitions queries by index residues. For a modulus M , each query index is assigned to a lane:

$$i \mapsto i \bmod M. \quad (1)$$

In the experiments below, we use mod30 lanes motivated by the modular wheel structure used in the companion pre-oracle filtering experiments.

3.3 Confidence-Aware Scheduling

Confidence-aware scheduling prioritizes queries using a model uncertainty score. For a linear classifier, decision-margin confidence provides a simple proxy. This connects readout scheduling to active learning and uncertainty sampling [2], but the goal here is not model retraining; the goal is controlled inference.

3.4 Hybrid Scheduling

Hybrid scheduling combines modular coverage and confidence prioritization. Queries are split into modular lanes, ranked within each lane by confidence, and interleaved across lanes. This provides a mechanism for balancing uncertainty targeting with structured coverage.

4 Stopping Rules and Coverage

4.1 Accuracy-Based Stopping

A basic stopping rule halts evaluation when a target predictive threshold is reached:

$$\text{Acc}(\pi_{1:k}) \geq \tau. \quad (2)$$

4.2 Coverage

Coverage measures how many structural lanes have been visited. For a set of modular lanes $\mathcal{L}$ and evaluated indices $\pi_{1:k}$, define

$$\text{Cov}(\pi_{1:k}) = \frac{|\{i \bmod M : i \in \pi_{1:k}\}|}{|\mathcal{L}|}. \quad (3)$$

Coverage captures structural completeness of evaluation.

4.3 Coverage-Constrained Stopping

We introduce a joint stopping rule:

$$\text{Acc}(\pi_{1:k}) \geq \tau \quad \text{and} \quad \text{Cov}(\pi_{1:k}) \geq \gamma. \quad (4)$$

This rule enforces both predictive performance and modular coverage. It is expected to require more queries than accuracy-only stopping, but provides a reproducible coverage guarantee.

5 Empirical Results

We evaluate readout policies on a 20 Newsgroups classification task using a fixed classifier. The experiments compare random, modular, confidence-aware, and hybrid scheduling policies. All results are interpreted as classical readout-control experiments, not as changes to the underlying model or QOS algorithm.

5.1 Accuracy vs. Coverage

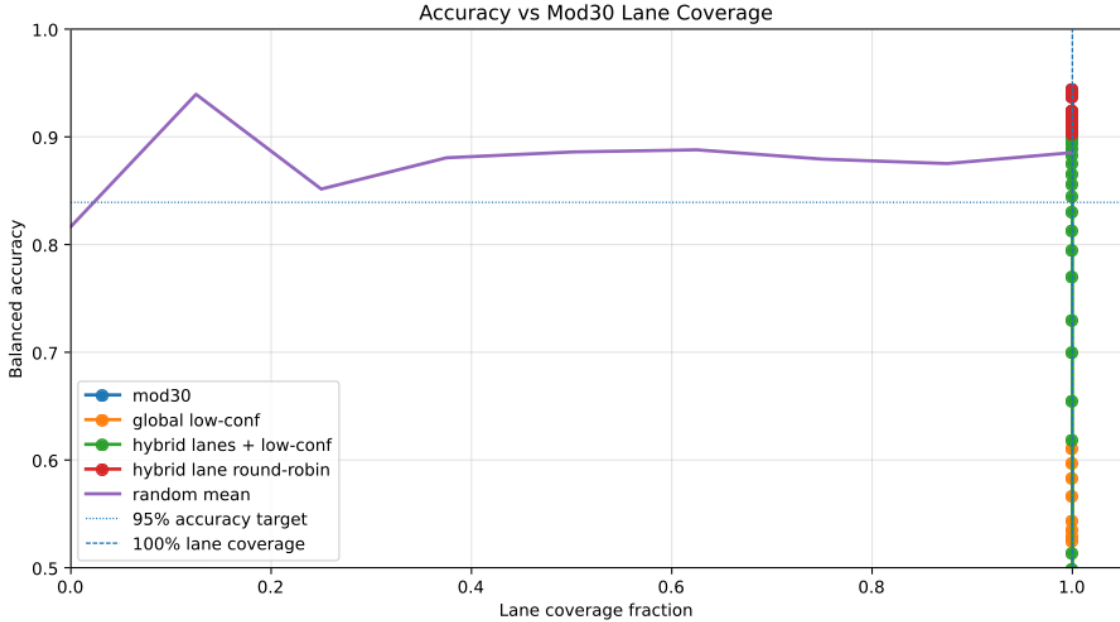


Figure 1: Balanced accuracy versus mod30 lane coverage. The figure illustrates that predictive behavior can saturate before full modular coverage is achieved. This motivates coverage-constrained stopping rules rather than accuracy-only stopping.

Figure 1 shows a central structural effect: accuracy can reach a useful level before all modular lanes are covered. This means that accuracy alone is not sufficient when a readout policy is expected to provide structural coverage.

5.2 Hybrid Scheduling Coverage

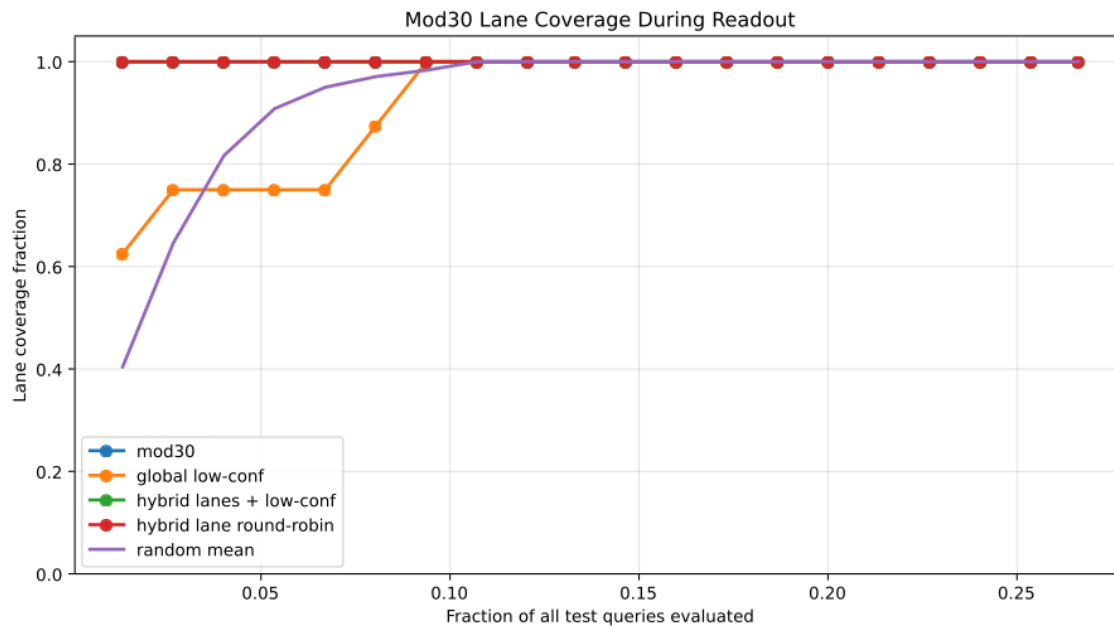


Figure 2: Mod30 lane coverage during progressive readout. Hybrid scheduling preserves structured coverage while incorporating confidence-aware ordering.

Figure 2 shows how modular and hybrid policies move through the readout space. The hybrid policies provide a reproducible way to distribute evaluation across lanes while still allowing confidence-aware prioritization within lanes.

5.3 Pareto Frontier

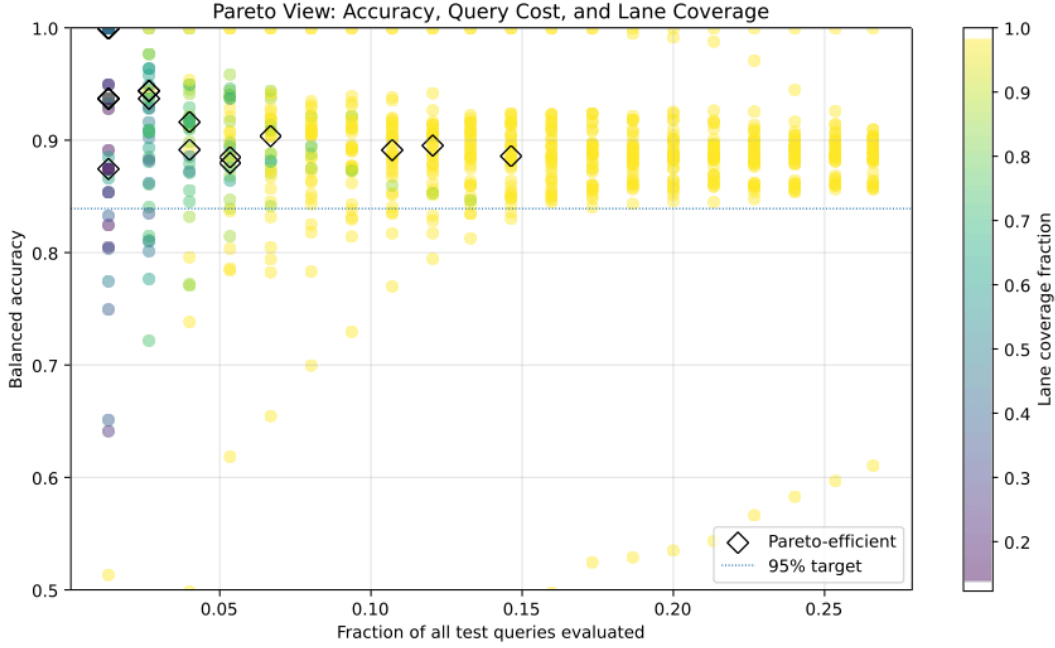


Figure 3: Pareto view of readout policies. Each point represents a candidate policy at a particular evaluation fraction. The highlighted Pareto-efficient policies define tradeoffs between query cost, balanced accuracy, and lane coverage.

Figure 3 summarizes the policy design space. No single policy dominates across all objectives; different operating points trade query cost, accuracy, coverage, and distributional stability.

6 Multi-Objective Policy Design

We define objective components for a candidate policy point:

$$C_{\text{query}} = \text{fraction of queries evaluated}, \quad (5)$$

$$C_{\text{acc}} = 1 - \frac{\text{Acc}(\pi_{1:k})}{\text{Acc}_{\text{full}}}, \quad (6)$$

$$C_{\text{cov}} = 1 - \text{Cov}(\pi_{1:k}), \quad (7)$$

$$C_{\text{dist}} = \text{distributional shift}. \quad (8)$$

A weighted objective is then

$$J = w_q C_{\text{query}} + w_a C_{\text{acc}} + w_c C_{\text{cov}} + w_d C_{\text{dist}}. \quad (9)$$

This follows the general pattern of cost-sensitive decision-making [1], but here the costs are applied to inference scheduling rather than supervised training.

Pareto-efficient policies are those for which no objective can be improved without worsening another. The resulting frontier provides an explicit design surface for selecting readout policies under resource constraints.

7 Discussion

The experiments support three observations. First, readout can be structured: deterministic policies provide reproducible evaluation orderings. Second, coverage matters: accuracy-only stopping can halt before all modular regions are visited. Third, policy choice is multi-objective: the optimal readout strategy depends on whether the user prioritizes query cost, predictive behavior, coverage, or distributional stability.

This framing is compatible with QOS-style workflows. A QOS pipeline may reduce or represent classical data efficiently, while a readout policy controls how sparse test queries are evaluated after model construction. The present work does not modify QOS internals, oracle construction, or quantum complexity.

8 Limitations

The experiments use simple modular partitions and classifier confidence proxies. Results may depend on dataset structure, model calibration, and the choice of coverage definition. Future work could replace index-based modular lanes with semantic clusters, learned partitions, or task-specific coverage metrics.

9 Conclusion

We introduced a framework for structured readout scheduling, coverage-aware stopping, and multi-objective policy optimization. The main contribution is to treat readout as a controllable inference process. This provides a reproducible way to reason about when to stop evaluation, what structural coverage is required, and how to select policies under competing objectives.

References

- [1] Charles Elkan. The foundations of cost-sensitive learning. In *IJCAI*, 2001.
- [2] Burr Settles. Active learning literature survey. Technical report, University of Wisconsin-Madison, 2010.
- [3] H. Zhao. Exponential quantum advantage in processing massive classical data. *arXiv preprint arXiv:2604.07639*, 2026.